Cinco Computer Consultants

Infrastructure Design Document

Barron Vaughn
bvaughn5@nebraska.edu
Lewis Rokke
lrokke2@nebraska.edu

University of Nebraska—Lincoln

Spring 2026
Version 6.0

This document details the features and components of the updated Cinco Computer Consultants invoicing subsystem infrastructure. It outlines a transition to an object-oriented model capable of persisting input data to a relational database, as well as the implementation of a custom data structure designed to organize financial records. This document is organized in a manner that allows the system design to be understood and reproduced by other developers.

Revision History

Version	Description of Change(s)	Author(s)	Date
1.0	Initial draft of this design document	B. Vaughn & L. Rokke	2026/2/6
2.0	Updates to sections 1, 2, & 3	B. Vaughn & L. Rokke	2026/2/20
3.0	Updated Class & Design Information	B. Vaughn & L. Rokke	2026/3/13
4.0	Updated Database Information	B. Vaughn & L. Rokke	2026/4/3
5.0	Data Structure Design	B. Vaughn & L. Rokke	2026/4/17
6.0	Data Structure Tesing	B. Vaughn & L. Rokke	2026/5/1

Table of Contents

REVISION HISTORY **2**

1. INTRODUCTION **4**

1.1 PURPOSE OF THIS DOCUMENT 5
1.2 SCOPE OF THE PROJECT 5
1.3 DEFINITIONS, ACRONYMS, ABBREVIATIONS 6
1.3.1 Definitions *6*
1.3.2 Abbreviations & Acronyms *6*

2. OVERALL DESIGN DESCRIPTION **6**

2.1 COMPONENT BREAKDOWN 6
2.1.1 Electronic Data Interchange *6*
2.1.2 Object Hierarchy Design *7*
2.1.3 Reporting *7*
2.1.4 Data Persistence *7*
2.1.5 Data Structure *7*
2.2 ALTERNATIVE DESIGN OPTIONS 7

3 DETAILED COMPONENT DESCRIPTION **8**

3.1 DATABASE DESIGN 9
3.1.1 Component Testing Strategy *10*
3.2 CLASS/ENTITY MODEL 11
3.2.1 The Item Hierarchy *11*
3.2.2 Business Entities *12*
3.2.3 System Drivers *14*
3.2.4 Component Testing Strategy *15*
3.3 DATABASE INTERFACE 15
3.3.1 Data Retrieval *15*
3.3.2 Connection Management *16*
3.3.3 Data Manipulation and Persistence *16*
3.3.4 Data Validation and Null Handling *16*
3.3.5 Data Integrity *16*
3.3.6 API Methods *17*
3.3.7 Component Testing Strategy *19*
3.4 DESIGN & INTEGRATION OF A SORTED LIST DATA STRUCTURE 19
3.4.1 Component Testing Strategy *21*

4. CHANGES & REFACTORING **21**

1. Introduction

Cinco Computer Consultants (CCC) is a regional computing services company providing support to businesses including equipment and other services such as training, consultations, and licenses. Currently CCC is redesigning its internal system that handles invoices and billing. The infrastructure overhaul detailed in this document was initiated by CCC CEO Dr. Steve Brule.

The primary function of the Invoice Subsystem is to process transactions for clients. Each invoice is uniquely identified and records creation date, customer information, salesperson, and a collection of billed items. To provide functionalities for all aspects of CCC's business model, the Invoice Subsystem will handle three distinct categories of items, each with specific billing rules.

Equipment	Physical products sold or leased to clients. Purchased equipment is billed by multiplying price per unit by requested quantity. Leased equipment is billed for a specific number of months, with the monthly cost calculated as the purchase price plus a 50% markup, amortized over a 36-month period. Clients are invoiced only for the initial month of the lease.
Services	Consultations, training, and troubleshooting provided by a CCC consultant. Services are billed using an hourly rate multiplied by the number of billable hours, plus a flat $125 service fee.
Licenses	Software, service hosting, and third-party subscriptions. Licenses are billed based on their effective dates. The cost is calculated by taking the number of active days divided by 365, multiplied by an annual license fee, plus a unique flat service fee.

Additionally, the Invoice Subsystem automatically calculates the appropriate taxes based on the items purchased.

Purchased Equipment	Subject to a 5.25% tax.
Leased Equipment	Taxed based on the total lease price. Leases under $2,000 are not taxed. Leases greater than $2,000 less than $7,000 incur a flat $175 tax. Leases of $7,000 or more receive a flat $350 tax.
Services	Subject to a 3.15% tax.
Licenses	Tax exempt.

1.1 Purpose of this Document

This document outlines the technical design details, data models, and business logic for the Cinco Computer Consultants (CCC) Invoice Subsystem. Its purpose is to serve as a blueprint for the system's design, ensuring that the software development is both maintainable and reproducible. The intended audience for this document includes software engineers, database administrators, application testers, and other technical staff responsible for maintaining or refactoring the application.

1.2 Scope of the Project

This project is limited to the development of the invoice subsystem for CCC. The application parses flat-file data and instantiates it into an object-oriented model. It then applies business logic to calculate billing and taxes, which are then used to generate summary reports. Finally, the system uses a database connection to persist these modeled objects into a relational database for future access.

Broadly, this project covers:

- Data interchange: Parsers to handle files and exporting data to formats such as XML and JSON.
- Object Modeling: Objects to model distinct business entities including *Persons*, *Companies*, and *Items.*
- Billing Logic: Implementations of company billing policies regarding equipment purchases, leases, and licensing fees, including relevant tax details.
- Reporting: Tools for creating reports summarizing information for all areas of the invoice subsystem, formatted for usability.
- Data Structure: A sorted list data structure used for organizing business records
- Database: Relational database modeling important aspects of the invoice system, including the ability to persist data

This project does *not* cover:

- Development of independent systems for areas such as inventory management or sales
- Data migration from legacy to updated systems
- Development of a Graphical User Interface (GUI)

1.3 Definitions, Acronyms, Abbreviations

1.3.1 Definitions

- Amoritization: Breaking up the cost of an asset by paying for it in installments over a specific length of time

1.3.2 Abbreviations & Acronyms

- CCC: Cinco Computer Consultants
- ER Diagram: Entity-Relationship Diagram
- EDI: Electronic Data Interchange
- JSON: JavaScript Object notation
- XML: eXtensible Markup Language
- UUID: Universally Unique Identifier
- OOP: Object Oriented Programming
- JDBC: Java Database Connectivity API
- ADT: Abstract Data Type

---

2. Overall Design Description

The CCC invoice system has been developed to provide several important capabilities. The system is ultimately designed to transition legacy flat-file data, including specific information about customers, companies, people, and products into an object-oriented ecosystem capable of handling billing logic, generating formatted reports, and persisting data to a database backed by a custom data structure.

2.1 Component Breakdown

The core components and design details of each aspect of the system are detailed below.

2.1.1 Electronic Data Interchange

The application uses dedicated loading and persistence classes to read flat CSV files, instantiating objects based on information from those files, and exporting them to the following formats:

- XML: Generated using an XStream library, applying the libraries' aliasing feature to ensure readability.

- JSON: Generated using a Gson library

2.1.2 Object Hierarchy Design

The Invoice Subsystem translates parsed data into an object-oriented model that represents CCC's real world operations. The system is designed to categorize entities into core business components: corporate clients (Companies), individual contacts (Persons), transactional records (Invoices), and billable items (Items). These models encapsulate all business logic, including the calculation of markups, amortizations, service fees, and taxes, meaning they act as the 'bridge' between data interchange and persistence.

2.1.3 Reporting

To provide insights into CCC's financial transactions, the Invoice Subsystem features a reporting function. This component processes the modeled invoice data and their associated line items to calculate final totals, taxes, and fees. The system is designed to generate three formatted summaries.

- Invoice Summary: Overview of all invoices alongside price totals
- Company Invoice Summary: Breakdown of invoice totals and transaction histories grouped by each client
- Detailed Invoice Report: List of all individual invoices, detailing each billed item, total costs, and specific terms (e.g. lease periods, license dates)

2.1.4 Data Persistence

The Invoice Subsystem uses a relational database hosted on a remote server. The Invoice Subsystem interfaces with this database via the Java Database Connectivity (JDBC) API. This API supports all functions necessary to insert, manipulate, and retrieve data to allow for the Invoice Subsystem to instantiate objects from database records while enforcing data integrity.

2.1.5 Data Structure

The Invoice subsystems include a built-in sorted array list. This array list will have three different ways it will be sorted using a comparator function, depending on which list the user needs. Interacting with this list will be simple for users as there will be public methods for adding, removing and finding elements in the list along with a few other convenience methods.

2.2 Alternative Design Options

During the design process of the Invoice Subsystem, several alternative implementation strategies were compared. The following are examples of alternative design approaches that were considered, and would be valid, but were ultimately decided against.

1. Tax Calculation Placement

The Invoice Subsystem could have used a single, centralized *TaxCalculator* utility class. The *Invoice* would pass each item to this calculator, which would then use *instanceof* checks to determine if the item was a *Lease*, *Purchase*, or *Service* and apply the appropriate tax calculation. Tax logic is instead strictly encapsulated within transactional classes (e.g. the *Lease* class inherently knows its own tax rules and is calculated in the 'getters' of that class). This was chosen as having a centralized calculator would have violated the principles of the object-oriented model.

2. Invoice Linking

The *Invoice* class could have simply stored the String UUIDs of the customer, salesperson, and items, exactly as they appear in an incoming CSV file. Instead, the *Invoice* class stores the actual *Company*, *Person*, and *InvoiceItem* objects themselves. If the *Invoice* only stored UUID strings, the reporting driver would have to constantly search through lists of companies and people to find the matching names each time it printed a line. Storing the actual objects inside the *Invoice* makes report generation significantly simpler.

---

3 Detailed Component Description

This section will provide a technical breakdown of the following components:

- The class entity hierarchy modeling all business entities while adhering to the Single Responsibility Principle
- The database interface used to facilitate communication data persistence
- The relational database used for data persistence, including database design and testing
- The Sorted List Data Structure used to organize and sort data
- Testing methodology and execution for all previous components to ensure reliability

3.1 Database Design

The Invoice Subsystem uses a MySQL relational database designed to persist object data while enforcing data integrity. The entity-relationship diagram for this database is included below.

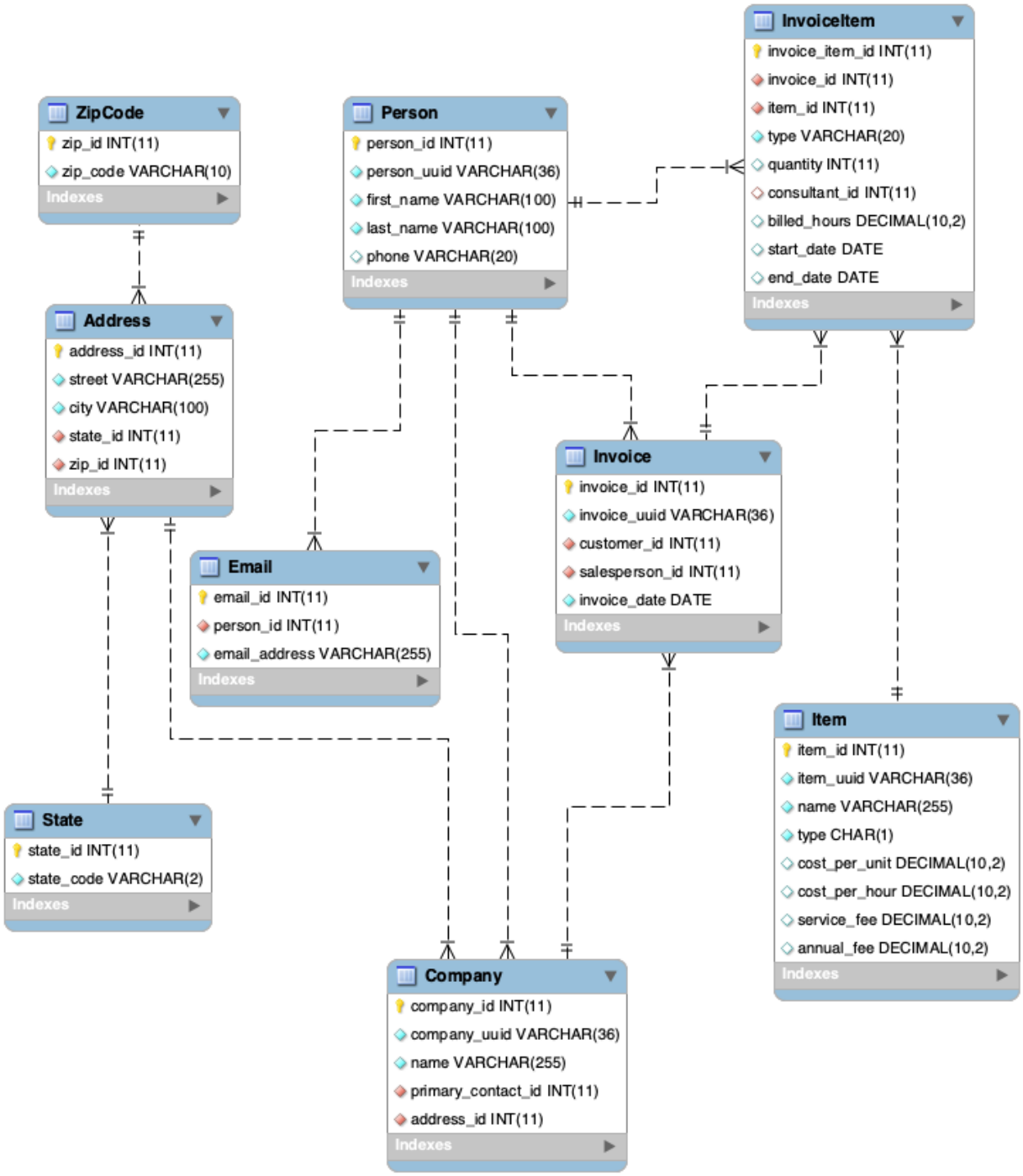


Figure 1: Entity Relationship (ER) diagram of the CCC Invoice Subsystem Relational Database

This database schema normalizes the business entities in accordance with our object-oriented model. Following best practices, all tables use auto-generated integers as primary keys. Extra identifiers (UUIDs) parsed from external data sources are stored as unique values.

To model CCC's billable items, the *item* and *invoice_item* tables use a *type* discriminator to determine the specific subtype of the record. The *item* table aggregates columns for all possible item attributes, allowing *Equipment*, *Services*, and *Licenses* to be stored within the same table. Similarly, the *invoice_item* includes details such as *quantity*, *term_in_months, billed_hours*, and effective dates to accommodate specific billing details.

Data integrity is enforced through strict foreign key constraints and orphaned record deletion. The *invoice* table maps transactions by linking a *customer_id* (foreign key to *company*) and a *salesperson_id* (foreign key to *person*). The *invoice_item* table acts as the associative entity mapping specific *item* records to an *invoice*. To prevent orphaned records *ON DELETE CASCADE* restraints are applied to dependent tables (e.g. deleting an *invoice* automatically clears its associated *invoice_item* records, and deleting a *person* automatically removed associated *email* records).

It's important to note that the InvoiceItem table has a *not null* type attribute as it will be set to either Purchase, Lease, BilledService, or BilledLicense.

3.1.1 Component Testing Strategy

The database model has been tested to ensure it accurately supports CCC's operations and maintains database integrity. This was primarily done by inserting substantial test data and executing a suite of 12 complex queries executed on mock test data designed to catch edge cases. A focus of this testing was validating that the schema successfully achieves Third Normal Form (3NF). The queries validated that all non-key attributes depend only on their respective primary keys, confirming that there are no transitive dependencies. This prevents update anomalies.

The queries:

- Used outer joins to calculate and verify total transaction counts for every corporate client.
- Aggregates subtotal changes for *item* records classified as *equipment* by calculating the product of quantity and unit cost directly within the query structure.
- Detect invalid data, such as duplicate *equipment* purchase records within a single *invoice*.
- Identify potential instances of employee fraud by querying for an *invoice* where the person acting as the *salesperson* is identically linked as the primary contact for the purchasing *company*.

Additionally, this component was tested by an external quality assurance team to ensure correctness.

3.2 Class/Entity Model

The CCC invoice system is built with an object-oriented class and entity model. Since CCC processes various transaction types, a standard procedural design would be fragile and overly complex. The Invoice Subsystem instead uses encapsulation and inheritance to directly model specific business entities. As introduced in Section 2, this design ensures that the reporting driver and database components interact with a uniform set of objects. The following section breaks down each area of the class/entity model, detailing each class and their relations to one another.

3.2.1 The Item Hierarchy

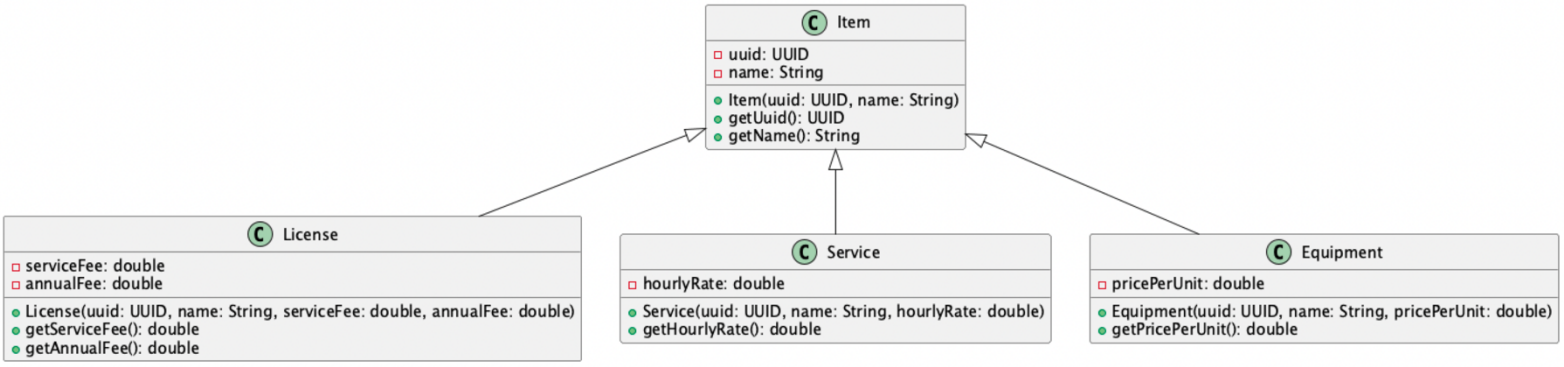


Figure 2: Item hierarchy showing the Item class as well as License, Service, and Equipment, which extend it.

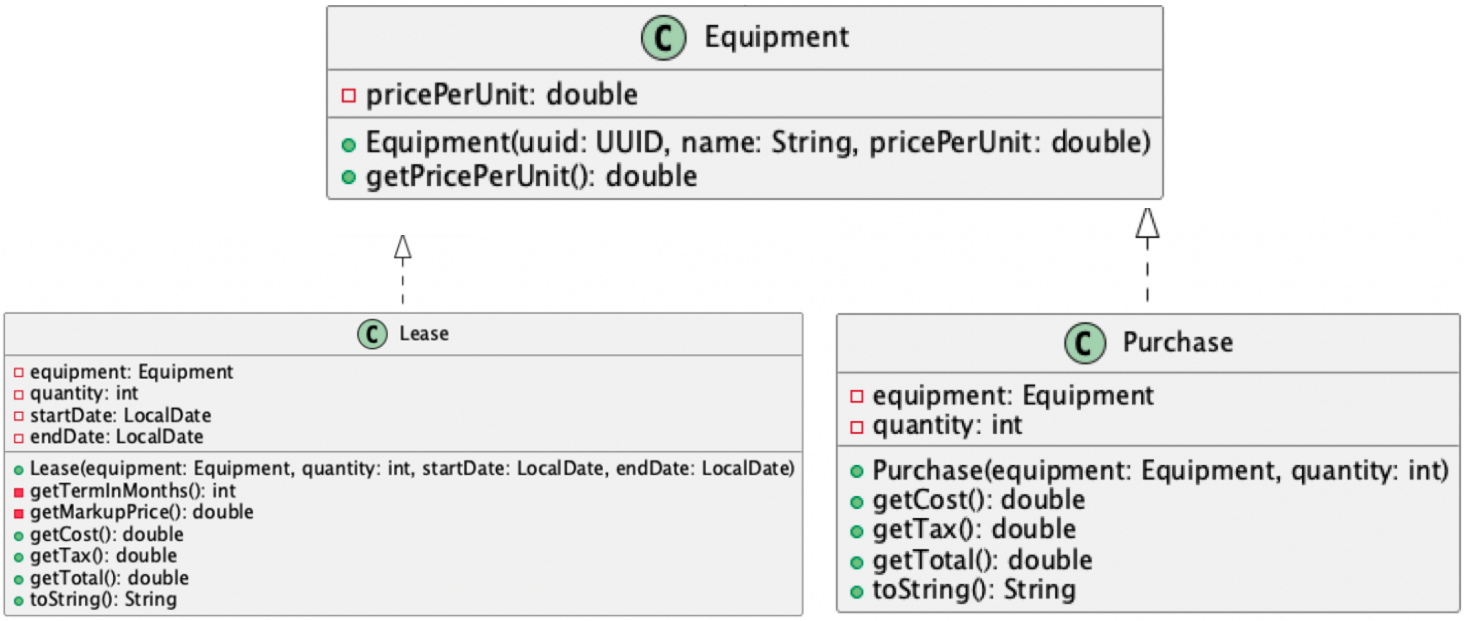


Figure 3: A focus on the equipment class as well as Lease and Purchase which extend it.

The Invoice Subsystem processes distinct item types: Licenses, Services, and Equipment, each operating under a different set of rules.

- *Item* (Base Class): Acts as the super class defining UUID and name across all entities.
- *Equipment*: Extends *Item* to encapsulate the specific physical item attribute of *pricePerUnit*. In addition, the *Equipment* class has two subclasses:
    - *Purchase*: Contains a piece of *Equipment* alongside the quantity purchased, including the application of a 5.25% sales tax.
    - *Lease*: Contains a piece of *Equipment*, the quantity, and the term in months. Includes the 36-month amortization logic and the tiered tax brackets required
- *Services*: Extends *Item* to handle *hourlyRate*, required for calculating costs
- *License*: Extends *Item* to handle software and subscriptions having states for *annualFee* and *serviceFee*.

By defining an abstract class to handle these subclasses, the system can now process lists mixed with item types, relying on overridden subclass rules to store and execute operations specific to subclasses.

3.2.2 Business Entities

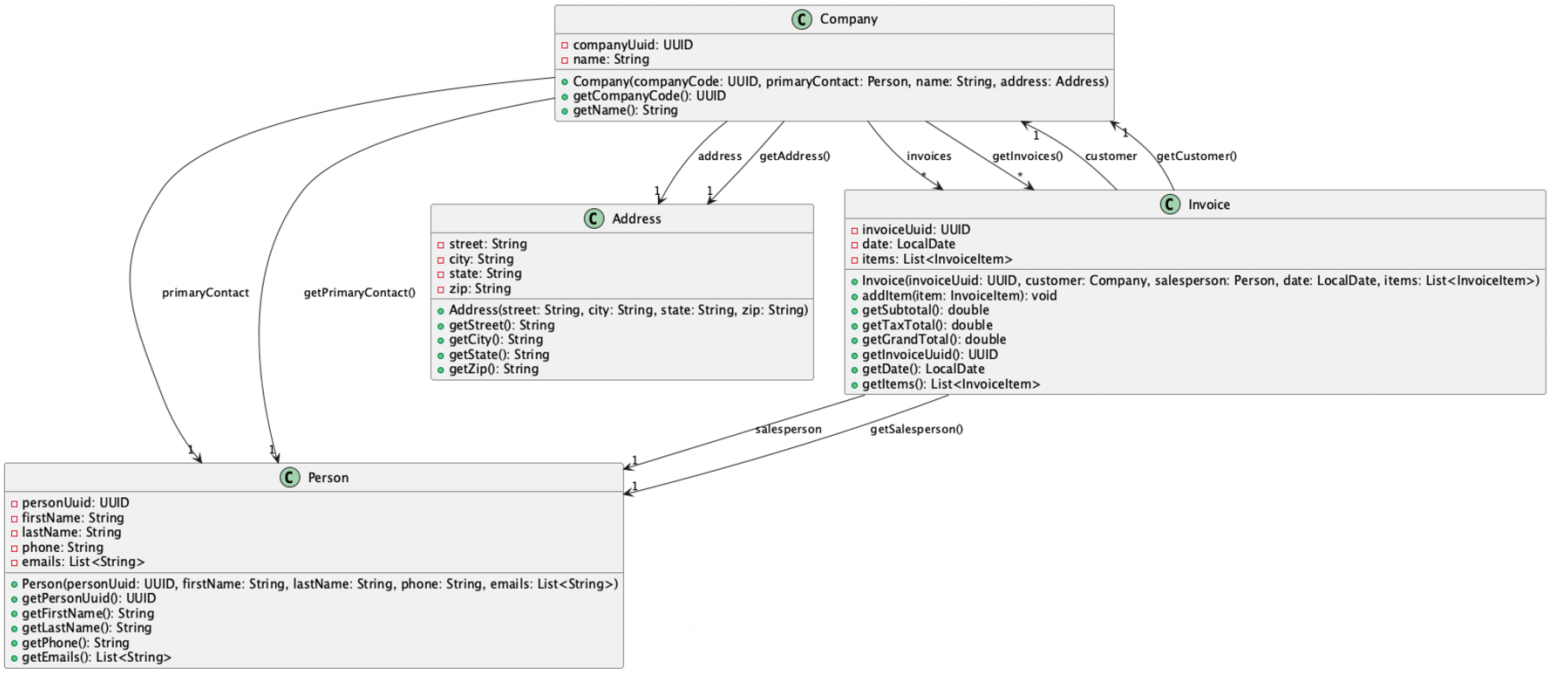


Figure 4: Class diagram modeling the core business entities and their relationships

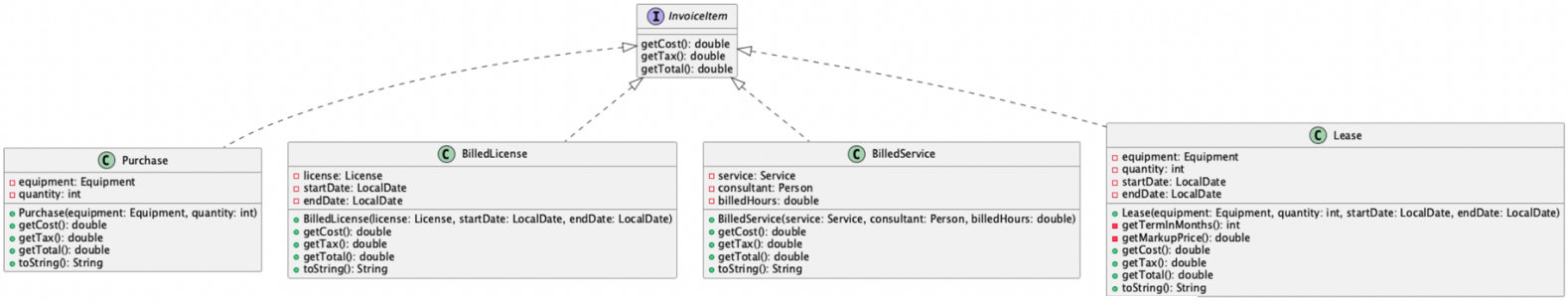


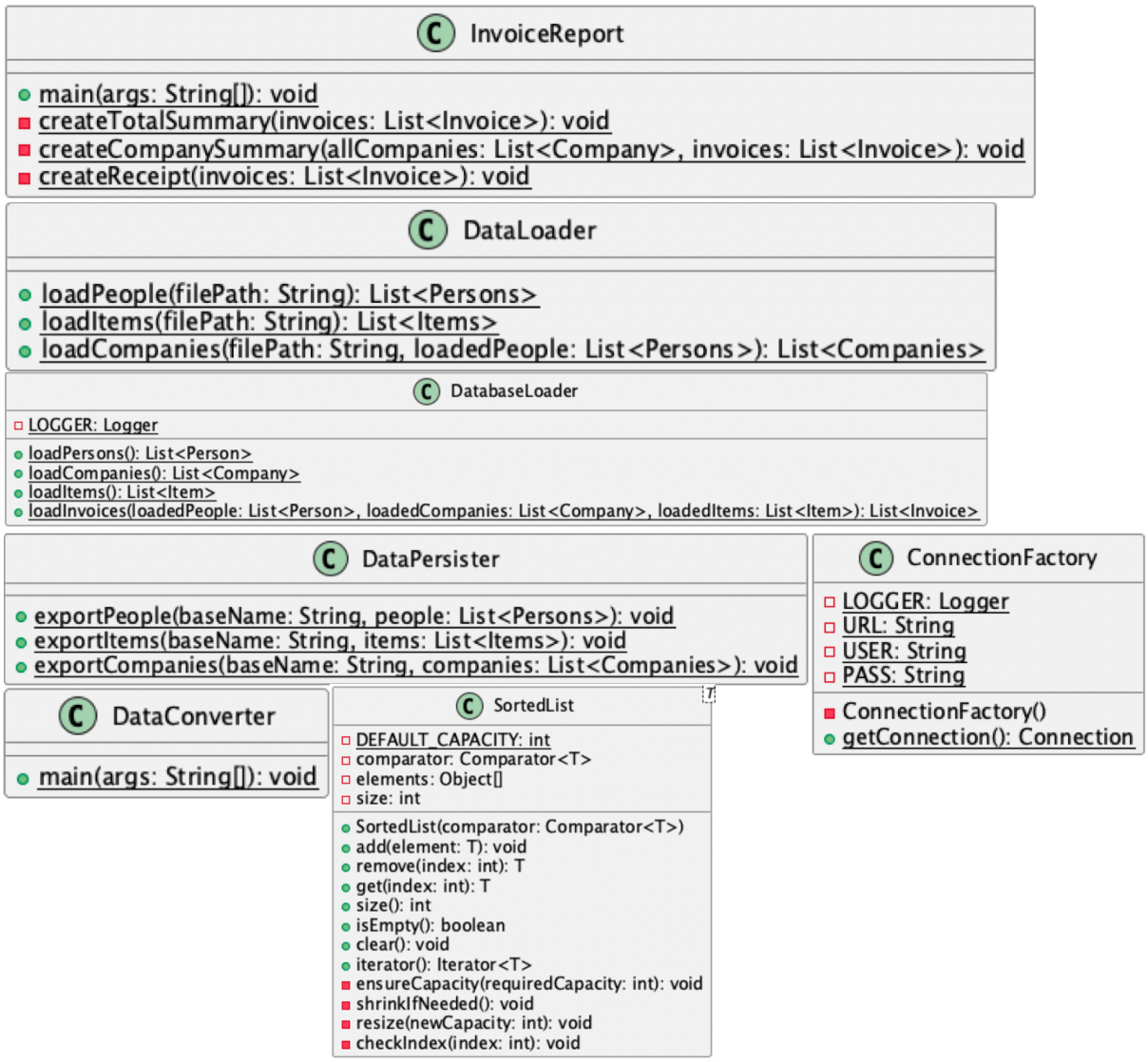
Figure 5: Class diagram showing the InvoiceItem interface and its implementing subclasses, Purchase, BilledLicense, BilledService, and Lease.

The Invoice Subsystem requires modeling of the corporate clients, individual contacts and transactional records.

- *Person*: Encapsulates individual contact details, including a list of email addresses.
- *Address*: Encapsulates street, city, state, and zip data, designed to be used by multiple classes.
- *Company*: Represents a corporate client, incorporating an *Address* object and a *Person* object to represent the primary contact, demonstrating a “has-a” relationship.
- *Invoice*: Central transactional entity, linking a customer (*Company*), a salesperson (*Person*), and a list of *InvoiceItems* (interface) .
- *BilledService*: An entity linking a *Service* to a *Person* performing the work and the number of billed hours.
- *BilledLicense*: An entity containing a *License* alongside its effective start and end dates.
- *InvoiceItem*: An interface that establishes the standard billing functions for all transactional objects. Requires *Purchase*, *BilledLicense*, *BilledService*, and *Lease* objects to provide calculations for getCost(), getTax(), and getTotal().

This design enables the Invoice Subsystem to process complex invoices and calculate totals without relying on specific underlying item types.

3.2.3 System Drivers



The Invoice Subsystem requires several drivers to execute necessary functionality. To adhere to the methodology of single responsibility, data loading, persistence, conversion, and reporting is separated.

- *DataLoader*: Parses flat CSV files, generating appropriate objects, making appropriate associations where necessary (e.g. linking a *Person* object to a *Company* contact field).
- *DataPersister*: Takes collections of objects and serializes them into XML and JSON file outputs.
- *DataConvertor*: The main driver of the application, retrieving flat files and calling previous methods to load and process them.

- *InvoiceReport*: The main driver for reporting, calling methods that generate the Invoice Summary, Company Breakdown, and Detailed Invoice List reports.
- *DatabaseLoader*: Connects to the MySQL database via JDBC to execute queries, retrieve records, and instantiate business objects.
- *ConnectionFactory*: Responsible for creating the connection to the database, providing a single private method to open the connection.
- *SortedList*:

3.2.4 Component Testing Strategy

The main approach for validating the class hierarchy and business entity model utilizes the JUnit testing framework. Testing focuses mostly on verifying business logic, specifically the accuracy of billing and tax calculations within transactional objects. Two test suites were developed to verify different layers of the model:

- *EntityTests*: Contains unit tests for each possible transaction type: *EquipmentPurchase*, *EquipmentLease*, *Service*, *License*. These tests verify that the *getCost()*, *getTax(),* and *getTotal()* methods return accurate values for individual items.
- *InvoiceTests*: Contains integration tests that aggregate multiple *InvoiceItem* objects into a single *Invoice* to verify the accurately of calculations and object associations across mixed lists

Additionally, testing data was generated using tools such as Mockaroo and generatedata as well as created manually to ensure coverage of all business rules. Integrated test cases ensure quality as they are designed to target all logical branches and edge cases within billing requirements. For example, specific test cases were created to trigger the different tax brackets for leased equipment (under $2,000, between $2,000 and $6,999.99, and $7,000 or above).

Alongside internal JUnit suites, the compiled object model and business logic was submitted to an external quality assurance team that verified functionality and accuracy.

3.3 Database Interface

The CCC Invoice Subsystem uses the Java Database Connectivity (JDBC) API to interface with the MySQL database detailed in previous sections. This interface serves as the companies means of transition away from flat-file parsing and to a more scalable platform. The API is centralized primarily within the *InvoiceData* class, which acts a bridge between the business logic and database schema.

3.3.1 Data Retrieval

To populate the Java object model, the application executes SQL queries using standard joins to aggregate related data. Instead of cluttering the driver class, data

instantiation is handled by dedicated factory methods. For example, a *buildPersonFromResultSet(ResultSet rs)* helper method parses the current row of data and instantiates a *person* object. This abstracts the SQL logic away from the core business entities.

3.3.2 Connection Management

To handle resources in a secure manner, a *ConnectionBuilder* helper class was implemented. This factory stores the database URL, username, and password credentials, returning a *connection* object when requested. This prevents redundant code.

3.3.3 Data Manipulation and Persistence

The InvoiceData class provides a group of methods (e.g. *addPerson()*, *addInvoice()*, *removeCompany()*) to modify database records. When persisting new data, the API uses only *PreparedStatement* objects, preventing injection attacks and ensuring data types are mapped correctly.

3.3.4 Data Validation and Null Handling

The API is designed to distinguish between application-level errors ('bad data') and valid empty states. If a method receives an inherently invalid argument, attempting to save a null object or passing an empty string as a UUID for example, the Java API intercepts this and throws an *IllegalArgumentException* before a database connection is even opened. Beyond this, the API actively prevents other forms of bad data by checking specific numbers such as quantities, billable hours, and item costs are within bounds.

The API also enforces field requirements based on the specific item type being processed. Because the database uses single inheritance with nullable columns, the Java methods verify that all required data is present prior to being executed. For example, it ensures that a lease or license transaction contains valid start and end dates, and that a billed service includes a valid consultant ID and billable hours. If the API receives *null* for a parameter that is optional, such as *consultant_id* for a standard equipment purchase, the *PreparedStatement* safely handles this using *setNull(index,java.sql.Types.VARCHAR)* to ensure that the database correctly resolves this to a null value rather than throwing a syntax error.

3.3.5 Data Integrity

When records are removed via the API, data integrity is enforced to prevent orphaned records. The API handles these side effects by relying on the ON DELETE CASCADE constraints established during the database design phase. For example:

- Deleting a Person record automatically purges all associated email records
- Deleting an *invoice* removes all associated *invoice_item* records

- If a user attempts to delete a *company* that still has active invoices, the database will reject the operation due to foreign key constraints, and the Java API will catch the resulting *SQLException*.

3.3.6 API Methods

The following is a table detailing each API Method, its target entities, its expected input, its handling of bad data, and actions taken to ensure integrity in the event of an error.

API Method	Target Entity	Expected Input	Handing of 'bad' data	Integrity Actions
clearDatabase()	All Tables	n/a	n/a	Deletes all records in reverse dependency order.
addPerson()	*person*	Valid UUID & Name Strings	Throws exception on null UUID/Names.	None
addEmail()	*email*	Valid person UUID, email string.	Fails if *person_id* returns empty.	Binds the new email only to the parent person.
addCompany()	*company, address, state, zip_code*	Valid UUIDs, name, and location strings.	Throws an exception if contactUuid doesn't exist in the person table.	Checks for inserts missing state and zip records, inserts the address, then links to the new company.
addEquipment()	*item*	Valid UUID, name, and positive rate.	Throws an exception on null UUID/name.	Hardcodes the type column to 'E'.
addService()	*item*	Valid UUID, Name, and	Throws an exception on	Hardcodes the type column to 'S'.

		positive fees.	null UUID/name.	
addLicense()	*item*	Valid UUID, name and positive fees.	Throws an exception on null UUID/name.	Hardcodes the type column to 'L'.
addInvoice()	*invoice*	Valid UUIDs, existing customer and salesperson UUIDs, valid LocalDate	Throws an exception if customer or salesperson UUIDs are missing.	Converts LocalDate to java.sql.Date for database storage
addEquipment purchaseToInvoice()	*invoice_item*	Valid Invoice & Item UUIDs, positive quantity.	Throws an exception for negative quantities	Hardcodes the type column to 'Purchase', leaves consultant_id and dates as null.
addEquipment LeaseToInvoice()	*invoice_item*	Valid Invoice & Item UUIDs, positive units.	Throws an Exception if UUID lookups fail.	Hardcodes the type column to 'Lease', leaves consultant_id and dates as null.
addServiceTo Invoice()	*invoice_item*	Valid Invoice, Item, and Consultant UUIDs, positive hours.	Throws an exception if consultant UUID does not exist in *person*.	Hardcodes the type column to 'BilledService', leaves dates as null.
addLicenseTo Invoice()	*invoice_item*	Valid Invoice and Item UUIDs, valid	Throws an exception if dates are null or end date	Hardcodes the type column to 'BilledLicense',

		start/end LocalDate.	precedes start.	converts dates to java.sql.date.

3.3.7 Component Testing Strategy

The database interface was tested in a manner that ensured that: data was being persisted reliably, objects were getting instantiated accurately, and connections were being managed. Unit tests for the *ConnectionFactory* verified connections closed properly to prevent resource leaks, while *DatabaseLoader* retrieval methods were tested against a dataset to confirm SQL rows correctly mapped to Java objects using the *type* discriminator. InvoiceData CRUD operations were validated against expected outputs and edge case testing. These tests confirmed the API safely handles null values, rejects data that is out of bounds, and does not leave behind orphaned records in the event of table deletion. The log4j framework was used to track database connections and catch errors for debugging.

3.4 Design & Integration of a Sorted List Data Structure

To organize financial records and generate summary reports, the Invoice Subsystem uses a custom Sorted List Abstract Data Type (ADT). This data structure is designed to hold any number of objects while maintaining ordering.

The sorted list exposes a group of public methods to the user, including:

add(t element)	Inserts a new element into its correct position
remove(int index)	Removes and returns the element at a position
get(int index)	Retrieves the element at the specified position
size()	Returns the number of elements in the list
clear()	Empties the list

To maximize its compatibility, it is constructed using *Comparator<T>,* allowing the application to instantiate multiple instances of the list with different sorting rules. The class implements the *Iterable<T>* interface which provides a custom *Iterator* that allows the list to be traversed using enhanced for-loops.

Internally, the ADT is implemented as an array-based list. It uses a generic *<T>* parameter, meaning the array is initialized as an *Object[]* array and cast to *T[],* allowing the list to be indifferent to the type of data it holds, whether that be *Invoice* objects, *Company* objects, or any other supported entity, provided the *comparator* is supplied.

The list dynamically expands and contracts as objects are manipulated to optimize resource utilization. When an element is added to the array and it reaches its capacity, a new array is created with double the size of the current array. Existing elements are copied before the new element is inserted. When an element is removed and the number of elements falls below 25% of the current capacity, the internal array is halved.

Order is maintained only at the point of insertion. There is no *sort* method. When add() is called, the list iterates through the current elements using the provided comparator to find the index where the new element belongs. Once it is located, all subsequent elements are shifted one position to the right to create an opening, and the new element is inserted. All ties are broken by comparing the UUIDs of the objects. When remove() is called, the element at the target index is removed, and all subsequent elements are shifted one position to the left.

The following is a visual example of our table behaviors.

1. Initial State

Index	0	1	2	3	4	5	6	7
Data	InvA	InvB	InvD	InvE	null	null	null	null
Totals	$10	$30	$50	$80				

2. Insert InvC ($40)

Step 1 Comparator locates insertion point at index 2 ($30 < $40 < $50)

Step 2 Elements at index ≥ 2 shift one position right

Index	0	1	2	3	4	5	6	7
Data	InvA	InvB	->	InvD	InvE	null	null	null

Step 3 InvC written to now-vacant index 2

Index	0	1	2	3	4	5	6	7
Data	InvA	InvB	**InvC**	InvD	InvE	null	null	null
Totals	$10	$30	$40	$50	$80			

3. Dynamic Resize (array full)

Step 1 All 4 slots occupied — allocate new array with double capacity (8)

Step 2 Copy existing elements to new array

Step 3 Insert new element at sorted position

Old (cap = 4)

	InvA	InvB	InvC	InvD

New (cap = 8)

Index	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
Data	InvA	InvB	InvC	InvD	**InvE**	null	null	null

3.4.1 Component Testing Strategy

The sorted list ADT was tested using both targeted unit tests and reporting validation to ensure that ordering, resizing, and retrieval behaved correctly under normal as well as edge case conditions. A dedicated JUnit test class, *SortedListTests*, was created to verify the core methods (*add(), remove()*, etc.). Test cases were written to confirm that elements are inserted into the proper position according to the given comparator, that ties are resolved using UUID comparison where applicable, and that removal shifts remaining elements correctly without corrupting list order.

Test data was drawn from mock data inserted into the CCC database designed t force specific ordering outcomes. For example, this test data included duplicate totals, identical customer names, and varying invoice counts. The results of these tests confirmed that the list maintained correct order at insertion time, preserved stable behavior during iteration, and supported the three required orderings without using JDK collection sorting methods.

4. Changes & Refactoring

Early reviews prompted several refactors to improve style, documentation, and design. Class names were normalized to singular form, and class-level documentation was added across the project. The model was updated to use Java's built-in UUID consistently and to represent a company's contact with a Person object through composition. DataConverter was simplified to reduce responsibility, and the earlier type field was removed to better support an object-oriented design. Overall, these changes made the codebase cleaner and more maintainable.